

Agentic URL Shortener

Bug Fixes Encountered During Implementation

This document summarizes several real issues encountered while building and validating the .NET 8 agentic URL-shortener assessment. Each example shows the symptom, root cause, fix, and engineering lesson.

1. xUnit types were not recognized

Symptoms: Visual Studio reported errors such as:

```
The name 'Assert' does not exist in the current context
The type or namespace name 'Fact' could not be found
The type or namespace name 'IClassFixture<>' could not be found
```

Root cause: The test source files were using xUnit types without the xUnit namespace being available in the file, or the xUnit NuGet packages had not yet restored.

Fix: Add the xUnit namespace to the test files:

```
using Xunit;
```

Also verify that the test project contains the xUnit and Microsoft.NET.Test.Sdk package references, then restore NuGet packages and rebuild.

Lesson: When Fact, Assert, and IClassFixture all fail together, first verify the test framework reference/namespace rather than changing test logic.

2. Integration test could not find WaitingForApproval

Symptom: Three integration tests passed, but the greenfield workflow test failed with an assertion similar to:

```
Assert.Contains() Failure
Not found: "WaitingForApproval"
```

Root cause: ASP.NET Core serializes enum values numerically by default. The C# workflow status was correct, but the JSON could contain:

```
"status": 1
```

while the integration test expected:

```
"status": "WaitingForApproval"
```

Fix: Configure controllers to serialize enums as strings:

```
using System.Text.Json.Serialization;

builder.Services
    .AddControllers()
    .AddJsonOptions(options =>
    {
        options.JsonSerializerOptions.Converters.Add(
            new JsonStringEnumConverter());
    });
```

Result: Swagger and tests receive readable values such as Running, WaitingForApproval, Completed, Rejected, SafeStopped, and RolledBack.

Lesson: A failing integration assertion does not always mean business logic is wrong. Inspect the HTTP representation as well as the underlying C# value.

3. Interlocked.Increment could not use the Clicks property

Symptom: An earlier implementation attempted:

```
Interlocked.Increment(ref item.Clicks);
```

Root cause: A normal C# property cannot be passed as a ref argument in this form.

Prototype fix:

```
lock (item)
{
    item.Clicks++;
    item.LastAccessedAt = DateTimeOffset.UtcNow;
}
```

Lesson: AI-generated concurrency code still needs compilation and review. A sophisticated-looking API call can be invalid for the actual model design.

4. Span/stackalloc language compatibility issue

Symptom: Earlier generated short-code logic used stackalloc/Span-style code in an async flow and produced a language/compiler compatibility problem.

```
Span<byte> bytes = stackalloc byte[7];
```

Fix: Use a simple byte array that works cleanly with the .NET 8/C# 12 target:

```
byte[] bytes = RandomNumberGenerator.GetBytes(7);
```

Lesson: Prefer the simplest implementation compatible with the project's target framework instead of upgrading the language version merely to accept generated code.

5. Swagger redirect looked like a CORS failure

Symptom: Testing GET /{code} through Swagger displayed a message such as 'Failed to fetch' and suggested CORS as a possible reason.

Root cause: The endpoint intentionally returns an external HTTP redirect. Swagger executes requests through browser fetch behavior, which is not always a good way to demonstrate an external redirect.

Fix: Create the short URL through Swagger, copy the returned shortUrl, and open it directly in a browser tab.

Lesson: Do not immediately add CORS configuration because a generic Swagger error mentions CORS. First verify the intended endpoint behavior.

6. Analytics returned 404 for an example code

Symptom:

```
GET /api/urls/X7ab92K/analytics
-> 404 Not Found
```

Root cause: The example code was not created by the currently running application. The prototype uses an in-memory repository, so restarting the API also clears previously created short URLs.

Fix: First call POST /api/urls, copy the actual generated code, and use that same code for redirect and analytics testing.

Lesson: Distinguish missing runtime data from a code defect.

7. Retry must be bounded

Risk: An agent that repeatedly fails could retry forever if the orchestrator has no explicit retry boundary.

Fix: The orchestrator limits attempts:

```
const int MaxAttempts = 2;

for (var attempt = 1; attempt <= MaxAttempts; attempt++)
{
    // Execute agent.
    // Retry only within the configured boundary.
}
```

After the retry boundary, the workflow uses a controlled fallback where appropriate or rolls back/safe-stops.

Lesson: Retry policy is part of agentic-system governance. Agents should not have unlimited autonomy to repeat failed actions.

8. Human approval must not be bypassed

Potential bug: Marking the workflow Completed immediately after all automated agents finish would bypass the required human-in-the-loop control.

Correct behavior:

```
run.Status = WorkflowStatus.WaitingForApproval;
```

Only the approval endpoint may subsequently transition the run to Completed or Rejected.

Lesson: Successful automated execution and authorization to finalize a change are different concerns.

9. Empty or unsafe requirements must stop before execution

Risk: Agents should not execute against an empty requirement or an instruction that violates the prototype's change-control policy.

Fix: The entry policy gate validates the request before the graph executes. Invalid input results in SafeStopped rather than allowing agents to guess.

Lesson: Guardrails should sit around the agent workflow, not depend entirely on each individual agent behaving correctly.

10. How these bugs demonstrate engineering judgment

These fixes show a useful AI-assisted development pattern: generate or accelerate implementation with AI, but compile the code, run unit and integration tests, inspect HTTP behavior, identify the actual root cause, make the smallest appropriate fix, and re-run validation.

A concise interview explanation is:

AI helped accelerate implementation, but I treated generated code as a proposal. During validation I found issues involving test-framework references, JSON enum serialization, C# ref semantics, language compatibility, Swagger redirect behavior, and runtime in-memory state. I fixed the root causes and re-ran the relevant tests rather than assuming the generated code or the first error message was correct.

11. Recommended validation sequence after a fix

1. Build -> Rebuild Solution
2. Test -> Test Explorer -> Run All
3. Confirm UnitTests are green
4. Confirm IntegrationTests are green
5. Start the API
6. Run the Swagger scenarios
7. Inspect workflow audit trail and metrics
8. Re-test failure/retry/fallback/rollback scenarios